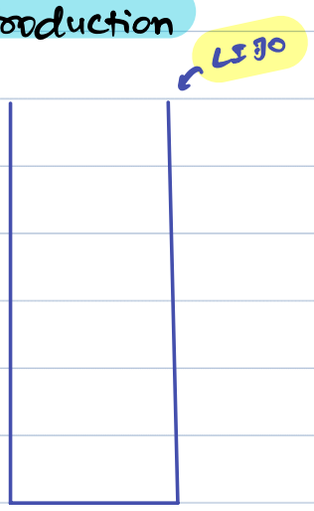
Today's agenda

- Queue basics
- Reverse first k ele in Queue
- implement queue using stack
- Kth number using only 1 & 2

AlgoPrep

// Introduction



↳ First In First Out (FIFO)

Real life ex:

(i) Line

(ii) task Scheduling

AlgoPrep

Syntax:

↳ Queue <Integer> q : new LinkedList <>();

name

Operations:

(i) q.add (x) → insert x at the end of queue.

(ii) q.remove() → delete element from front.

(iii) q.Peek () → return the front element.

(iv) q.Size () → No. of elements in queue.

Q) Reverse first K elements

↳ Given a Queue, Reverse it's first K elements.

K=4

Ex: 3 10 2 12 19 6 8 10 14

Idea

↳ Put the first K elements in the Stack.

K=4

q: ~~3~~ ~~10~~ ~~2~~ ~~12~~ 19 6 8 10 14

q: 19 6 8 10 14 12 2 10 3

q: ~~10~~ ~~8~~ ~~8~~ ~~10~~ ~~14~~ 12 2 10 3 19 6

8 10 14

~~12~~
~~2~~
~~10~~
~~3~~

q: 12 2 10 3 19 6 8 10 14

Steps 1:

↳ Push K elements Queue → Stack.

Step 2:

↳ Put K elements back from Stack → Queue.

Step 3:

↳ Remove the first n-k elements from the front and add at the end.

AlgoPrep

//Psuedo code

```
Queue<> ReverseKelements (Queue <>q, int K){
    Stack <Integer> S = new Stack<>();
        int n= q.size();
        for (int i=0; i<K; i++){
            S.push(q.remove());
        }

        for (int i=0; i<K; i++){
            q.add(S.pop());
        }

        for (int i=0; i<n-K; i++){
            q.add(q.remove());
        }

        return q;
}
```

T.C: O(N)

S.C: O(K)
↓
K≈N

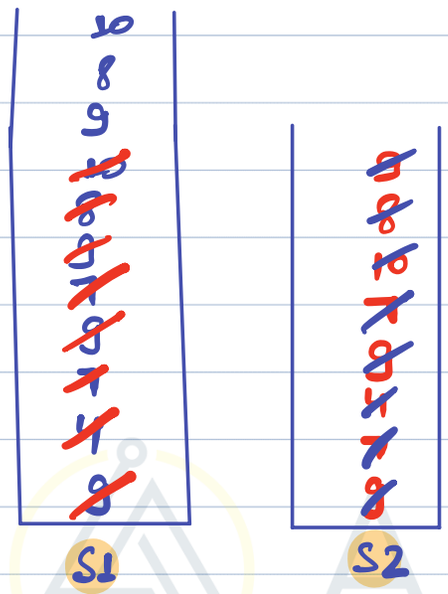
AlgoPrep

Q) Implement Queue using Stacks

FIFO → Queue, LIFO → Stacks

op: 5 4 7 9 rem rem() 8 10 rem rem 14 rem

↓ 5 ↓ 4 ↓ 7



//idea1 → add efficient

add(x): add x in S1.

↳ O(1)

remove(): → O(N)

(i) move n-1 elements S1 → S2

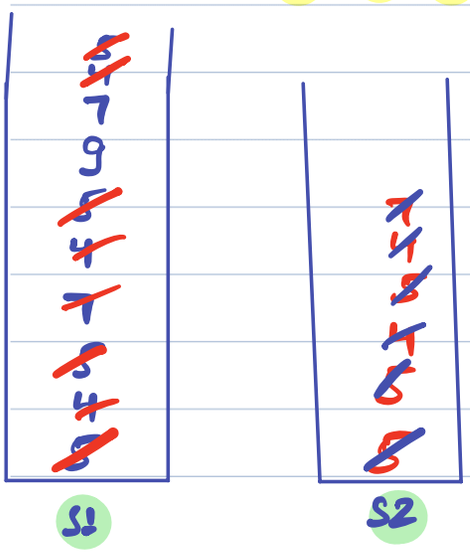
(ii) The remaining element of S1 is your ans.

(iii) move all elements from S2 → S1

//idea2 → remove efficient

op: 5 4 7 9 rem rem() 8 10 rem rem 14 rem

↓ 5 ↓ 4



add(x): → O(N)

(i) move all elements S1 → S2

(ii) add x to S1.

(iii) Put back all elements S2 → S1

remove(): → O(1)

↳ remove the top of S1.

AlgoPrep

Q) Kth Number

↳ Generate Kth number in series using digits 1 and 2 only.

K=5   1   2   11   12   21

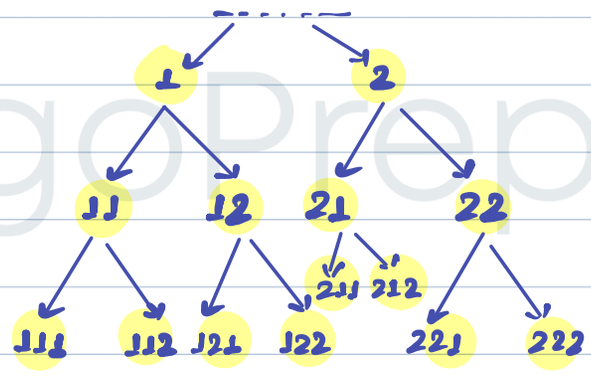
K=7   1   2   11   12   21   22   111

//idea

1 digit num

2 digit num

3 digit num



//Algorithm

K=5

Q: ~~1~~ ~~2~~ ~~11~~ ~~12~~ ~~21~~ 22 111 112 121 122

↳ ans = 21

//Psuedo code

```
String Kthnumber (int k) {
    Queue <String> q;
        q.add ("1");
        q.add ("2");

        String ans: " ";
        for (int i=1; i<=k ; i++) {
            String temp = q.remove();
            if (i==k) { ans = temp; }
            q.add (temp+ "1");
            q.add (temp+ "2");
        }

        return ans;
}
```

digit length

T.C: $O(k * n)$

S.C: $O(k)$

Break till 9:40 pm

Q)

↳ Generate Kth Palindrome number in series using digits 1 and 2 only

Note: Only consider even digit numbers.

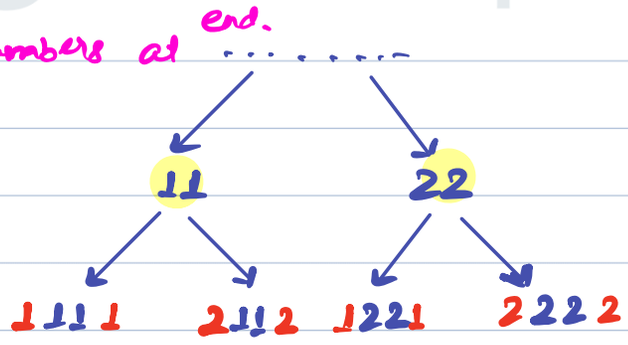
K=5 : 11 22 1111 1221 2112

//idea 1

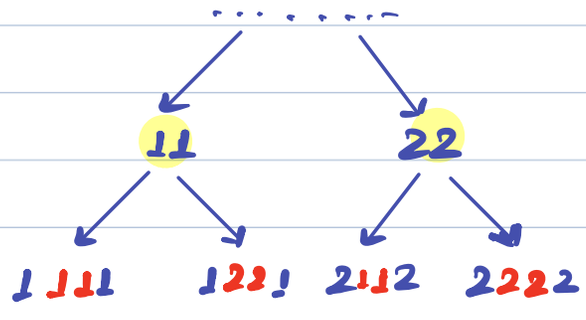
↳ Keep generating numbers using 1 and 2, check for even digit Palindrome and return the Kth one.

AlgoPrep

//idea 2 → insert same numbers at end.



→ insert same numbers in middle.



//Pseudo code

```
String KthNumber (int K) {
    Queue <String> q;

    q.add ("11");
    q.add ("22");
    String ans: " ";
    for (int i=1; i<=k; i++) {
        String temp= q.remove();
        if (i==k) { ans = temp; }
        String left = temp.SubString ( 0, (temp.length()-1)/2 );

        String right: temp.SubString ((temp.length()-1)/2, temp.length() )

        q.add ( left + "11" + right);
        q.add ( left + "22" + right);
    }
    return ans;
}
```

digit length

T.C: $O(K * n)$

S.C: $O(K)$

inc. exc.